

## Signals and Systems

### Computer Assignment 3: The Discrete Fourier Transform (DFT)

Department of Electrical Engineering  
Sharif University of Technology  
Instructor: Dr. Hamid Behroozi

Semester: Spring 1405

Contact: If you have any questions, please contact the TA via Telegram(@i\_NMA1Ex) or Bale(@Motenaferamazbale).

#### Submission Guidelines

1. Report Format: Submit your assignment report strictly in PDF format. The report must include all outputs, plots, code explanations, analytical derivations, and thorough results analysis.
2. Code Structure: Submit your complete code as a single .m file. Different sections of the assignment must be clearly partitioned using MATLAB section markers (%%).
3. Custom Functions: Any custom functions you write must be placed exclusively at the bottom of the main file in a dedicated section titled Functions. Avoid creating multiple separate MATLAB files.
4. Archiving: Archive all necessary files into a single .zip or .rar package labeled strictly as: StudentNumber\_CHW3.zip.

#### Grading Criteria

- Organization, structural readability, and depth of analysis in the report.
- Correct mathematical implementation of algorithms and error-free code execution.
- Proper use of matrix vectorization instead of explicit indexing loops.
- Quality of visual plots (including titles, precise axis labels mapping to physical units, and legends).

### 1. Algorithmic Implementation & Computational Complexity of the DFT

The N-point Discrete Fourier Transform (DFT) of a sequence  $x[n]$  of length N is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N-1$$

### 1.1 Direct Evaluation vs. Matrix Matrix Vectorization

In this section, you will explore the computational mechanics of the DFT by building your own transform engines without relying on MATLAB's built-in fft function.

- (a) Write a custom function named myDFTLoop that computes the N-point DFT of an input vector  $x[n]$  using nested loops.
- (b) Write a second function named myDFTMatrix that performs the exact same computation but completely eliminates loops. Construct the  $N \times N$  DFT matrix ( $W_N$ ) containing the twiddle factors ( $W_N^{kn} = e^{-j\frac{2\pi}{N}kn}$ ), such that the transform is computed via a single matrix-vector multiplication: ( $X = W_N x$ ).

### 1.2 Execution Profiling & Asymptotic Complexity

- (c) Generate a random complex sequence of length  $N = 256$ . Compute its DFT using myDFTLoop, myDFTMatrix, and MATLAB's intrinsic fft. Verify that the execution outputs match by computing the norm of their differences.
- (d) Using tic and toc, measure the execution time of all three methods for varying sequence lengths  $N \in \{64, 128, 256, 512, 1024, 2048\}$ . Present your results clearly in a comparative table.
- (e) Plot the execution times against  $N$  on a log-log scale. In your report, provide a detailed analysis explaining the differences in execution speeds based on computational complexity ( $\mathcal{O}(N^2)$  vs.  $\mathcal{O}(N \log N)$ ) and the architectural benefits of memory vectorization.

## 2. Spectral Sampling, Frequency Resolution, and Leakage

Because the DFT samples the continuous-time Fourier transform (DTFT) of a finite-duration signal at discrete frequencies ( $\omega_k = \frac{2\pi}{N}k$ ), truncating and windowing signals introduces structural changes to their spectrums.

Consider the continuous-time signal ( $x(t) = \cos(2\pi f_1 t) + 0.5 \cos(2\pi f_2 t)$ ), where  $f_1 = 100$  Hz and  $f_2 = 105$  Hz. This signal is sampled at a rate of  $f_s = 1000$  Hz over an interval of duration  $T$ .

### 2.1 The Effect of Signal Duration and Zero-Padding

- (a) Set the observation window duration to  $T = 0.1$  seconds, yielding  $N_1$  samples. Compute the  $N_1$ -point DFT using fft and plot its magnitude. Ensure the horizontal axis is mapped precisely to physical frequencies in Hertz (Hz), covering the interval from 0 to  $f_s$ . Can you visually distinguish the two distinct sinusoidal peaks? Explain why based on the concept of frequency resolution ( $\Delta f = 1/T$ ).

- (b) Take the same  $N_1$  samples from part (a) and pad them with zeros until the total sequence length reaches ( $N_{pad} = 1024$ ). Compute the 1024-point DFT and plot its magnitude. Does zero-padding improve the fundamental physical resolution of your spectrum? Explain what physical effect zero-padding has on the DTFT envelope vs. the DFT bins.
- (c) Now, change the observation window duration to  $T = 0.5$  seconds, yielding  $N_2$  samples without any zero-padding. Compute its  $N_2$ -point DFT and plot the resulting magnitude spectrum. Compare this plot with the figures from parts (a) and (b), and analyze the structural differences in resolving close spectral components.

## 2.2 Windowing & Spectral Leakage

- (d) Modify the signal parameters such that  $f_1 = 100$  Hz and  $f_2 = 123.5$  Hz. Sample the signal at  $f_s = 1000$  Hz for a duration of  $T = 0.2$  seconds. This creates a rectangular window truncation that does not align with a whole number of periods for  $f_2$ . Plot the magnitude of the DFT. Explain the appearance of the "sloped skirts" (spectral leakage) surrounding the peaks.
- (e) Apply a Hann window (hann) and a Hamming window (hamming) to your time-domain samples before calculating the DFT. Plot the modified magnitude spectra over the unwindowed spectrum in a single figure using subplot.
- (f) In your report, contrast the trade-offs between a rectangular window and tapered windows in terms of main-lobe width (spectral resolution) and side-lobe attenuation (spectral leakage suppression).

## 3. Linear Convolution via the DFT (Circular Overlap)

By the duality property of the Fourier transform, multiplication in the discrete frequency domain corresponds to circular convolution in the time domain:

$$x[n] \odot_N h[n] \iff X[k] \cdot H[k]$$

### 3.1 Circular vs. Linear Convolution Mechanisms

- (a) Analytically demonstrate that an  $N$ -point circular convolution of two finite sequences  $x[n]$  (of length  $L$ ) and  $h[n]$  (of length  $M$ ) yields the exact same numerical values as their linear convolution if and only if  $N \geq L + M - 1$ .
- (b) Define the following two discrete sequences within their appropriate index ranges:

$$x[n] = n(u[n] - u[n - 8])$$

$$h[n] = (0.8)^n(u[n] - u[n - 6])$$

Compute their exact linear convolution ( $y_{lin}[n]$ ) manually or using MATLAB's native conv function to use as a baseline.

- (c) Calculate the N-point circular convolution of  $x[n]$  and  $h[n]$  purely in the frequency domain using the relation  $y_{circ}[n] = \mathcal{F}^{-1}\{\mathcal{F}\{x[n]\} \cdot \mathcal{F}\{h[n]\}\}$ . Execute this operation via the fft and ifft commands for  $N = 8$  and  $N = 14$ .
- (d) Plot the baseline linear convolution alongside both the  $N = 8$  and  $N = 14$  circular convolution sequences using the stem function in a single figure window. Highlight the sections where time-domain aliasing occurs when  $N = 8$ , and explain the result using your derivations from part (a).

#### 4. Dual-Tone Multi-Frequency (DTMF) Touch-Tone Decoding

Dual-Tone Multi-Frequency (DTMF) signaling is utilized in telephony systems to identify dialed digits. When a keypad button is pressed, it generates a unique signal consisting of the sum of two pure sinusoids: one from a lower frequency group (row) and one from a higher frequency group (column).

| Keypad | 1209 Hz | 1336 Hz | 1477 Hz |
|--------|---------|---------|---------|
| 697 Hz | 1       | 2       | 3       |
| 770 Hz | 4       | 5       | 6       |
| 852 Hz | 7       | 8       | 9       |

##### 4.1 Signal Generation & Spectral Diagnostics

- (a) Write a MATLAB script to synthesize the DTMF signal corresponding to dialing the digit "8". The signal should be sampled at  $(f_s = 8000 \text{ Hz})$  over a duration of 0.15 seconds. Plot this time-domain signal waveform over a reasonable sub-interval that displays its periodic characteristics.
- (b) Compute the 2048-point DFT of your synthesized signal. Apply fftshift to center the spectrum and plot the resulting magnitude versus the actual frequency axis across the range  $[-f_s/2, f_s/2] \text{ Hz}$ . Verify that the prominent spectral peaks align exactly with the row and column frequencies assigned to digit 8 in the standard specifications table.

##### 4.2 Automated Keypad Decoder Design

- (c) The course system provides a data file titled dtmf\_signal.mat. Load this file into your workspace. It contains an unknown DTMF vector  $x_{received}$  and its corresponding sampling rate  $fs_{received}$ .
- (d) Design an automated decoding algorithm using the DFT. Partition the continuous vector into distinct signal frames (representing individual dialed buttons separated by quiet intervals). For each active frame, compute its magnitude spectrum via the fft.
- (e) Write a custom classification function that detects the two dominant peak frequencies within the row and column spectral bands, matches them to the standard DTMF grid, and outputs the decoded numeric sequence directly to the command window.

- (f) Display the complete magnitude spectrum of the received signal as a 2D spectrogram (using a moving window DFT or the spectrogram function). Include this heat-map in your final report to visually illustrate how the frequency content evolves over time.

## 5. Two-Dimensional DFT and Image Spectrogram Processing

The 2D Discrete Fourier Transform extends the 1D spatial frequency mapping to matrices, converting spatial variations (images) into spatial frequency grids:

$$X[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x[m, n] e^{-j2\pi \left( \frac{km}{M} + \frac{ln}{N} \right)}$$

### 5.1 Spatial Patterns vs. 2D Spectral Topographies

- (a) Synthesize a  $256 \times 256$  binary image containing a repeating vertical grating pattern (alternating black and white bars every 8 pixels). Compute its 2D DFT using `fft2`, apply `fftshift`, and display its magnitude spectrum as an intensity image using `imagesc` or `imshow`.
- (b) Synthesize a second  $256 \times 256$  binary image containing a solid centered square of size  $32 \times 32$  pixels. Compute its centered 2D DFT magnitude spectrum and display it. Explain why a spatial square maps to a 2D cross-shaped sinc pattern in the frequency domain, noting the inverse relationship between spatial dimensions and spectral bandwidth.

### 5.2 Frequency-Domain Structural Filtering

- (c) Load a noisy color image of your choice using `imread`. Convert it to grayscale using `rgb2gray`. Introduce periodic noise into the image by adding a two-dimensional sinusoidal grid pattern:

$$n[x, y] = 0.4 \cos(2\pi(u_0x + v_0y))$$

where you choose fixed spatial frequencies  $u_0$  and  $v_0$ . Display the corrupted image.

- (d) Compute the centered 2D DFT of the corrupted image using `fft2` and `fftshift`. Plot its magnitude spectrum as a 3D surface plot using `mesh` or `surf` to easily spot the sharp, high-energy impulses caused by the periodic noise.
- (e) Design a 2D Frequency Notch Filter. Create a masking matrix that equals 1 everywhere except for localized clusters of zeros centered directly over the coordinates of the noise impulses. Multiply your shifted image spectrum element-wise by this notch mask.
- (f) Transform the filtered spectrum back into the spatial domain using `ifftshift` and `ifft2`. Retain only the real part of the result using `real`. Display the restored image alongside the original and corrupted versions within a single figure. Analyze the performance of

your filter, paying close attention to image sharpness, edge preservation, and any remaining artifacts.

## **6. Functions Section**

Place all your custom subroutines—myDFTLoop, myDFTMatrix, and your DTMF classification routine—below this line at the absolute end of your script file.